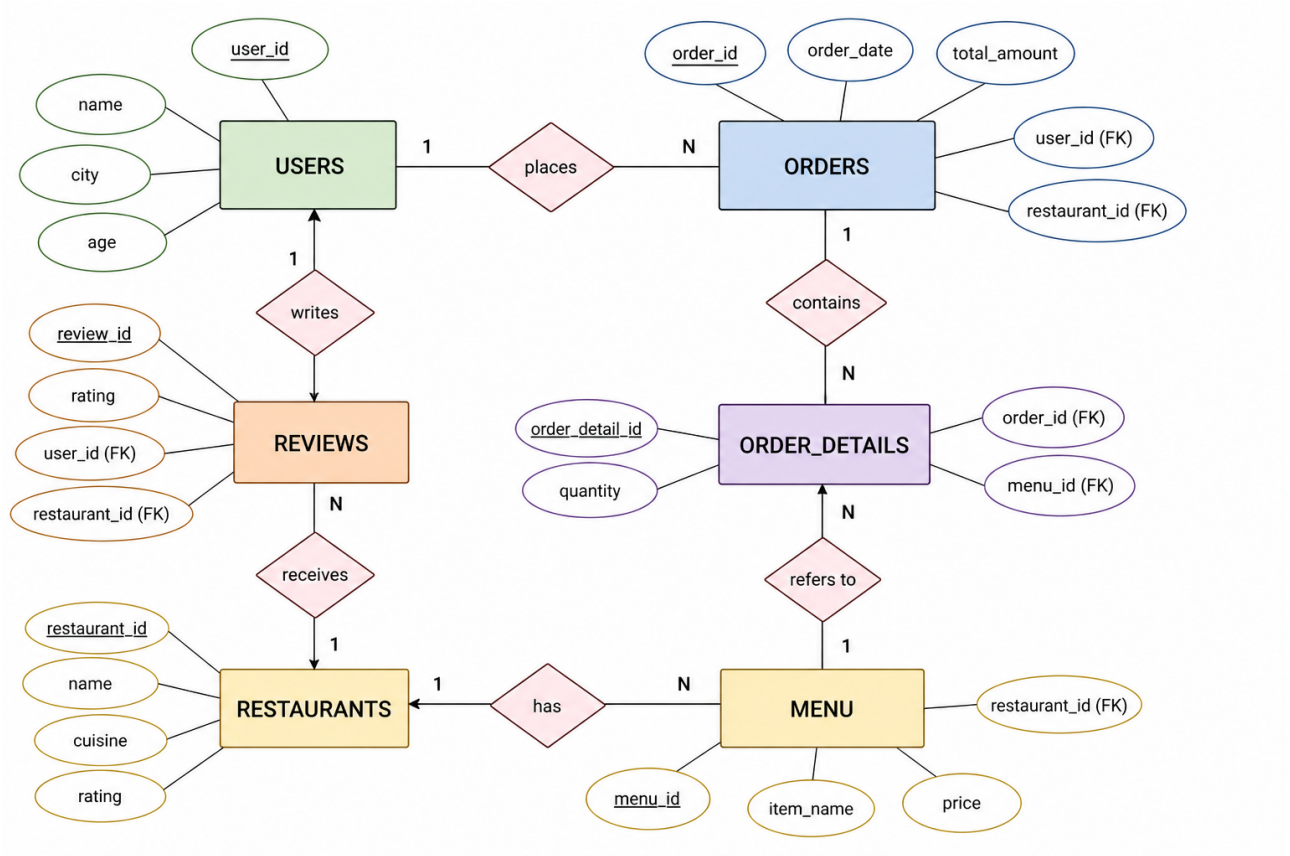
Shiva Prasad A M
1AY24CD051
4A-DS
CS(DS)
AIT.

DBMS Assignment:Designing a relational database for an online food ordering system and implementing SQL queries to retrieve, manipulate, and analyze data.

TABLE CREATION OF ENTITIES:

```
-- Users Table
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100),
    city VARCHAR(50),
    age INT
);

-- Restaurants Table
CREATE TABLE Restaurants (
    restaurant_id INT PRIMARY KEY,
    name VARCHAR(100),
    cuisine VARCHAR(50),
    rating DECIMAL(2,1)
);

-- Menu Table
CREATE TABLE Menu (
    menu_id INT PRIMARY KEY,
    restaurant_id INT,
    item_name VARCHAR(100),
    price DECIMAL(10,2),
    FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)
);

-- Orders Table
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    restaurant_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)
);

-- OrderDetails Table
CREATE TABLE OrderDetails (
    order_detail_id INT PRIMARY KEY,
    order_id INT,
    menu_id INT,
    quantity INT,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (menu_id) REFERENCES Menu(menu_id)
);

-- Reviews Table
CREATE TABLE Reviews (
    review_id INT PRIMARY KEY,
```

```
    user_id INT,
    restaurant_id INT,
    rating INT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)
);
```



SQL Queries with Questions:

1. Display all users from the Users table.
   SELECT * FROM Users;

2. List all restaurants with rating greater than 4.
   SELECT * FROM Restaurants WHERE rating > 4;

3. Show all menu items with price greater than 200
   SELECT * FROM Menu WHERE price > 200;

4. Find all users belonging to Bangalore.
   SELECT * FROM Users WHERE city = ‘Bangalore';

5. Retrieve all orders placed after ‘2024-01-10’
   SELECT * FROM Orders WHERE order_date > ‘2024-01-10';

6. Display restaurant names along with their cuisine type.
   SELECT name, cuisine FROM Restaurants;

7. Show all orders with total amount greater than 300.

```
SELECT * FROM Orders WHERE total_amount > 300;
```

8. Find the number of users in each city.
```
SELECT city, COUNT(*) AS total_users
FROM Users
GROUP BY city;
```

9. Display all menu items of a specific restaurant (restaurant_id = 1).
```
SELECT * FROM Menu
WHERE restaurant_id = 1;
```

10. Show all orders placed by user with user_id = 1
```
SELECT * FROM Orders WHERE user_id = 1;
```

11. Retrieve restaurant names and their average rating from Reviews.
```
SELECT r.name, AVG(rv.rating) AS avg_rating
FROM Restaurants r
JOIN Reviews rv ON r.restaurant_id = rv.restaurant_id
GROUP BY r.name;
```

12. Display total revenue generated by each restaurant.
```
SELECT restaurant_id, SUM(total_amount) AS revenue
FROM Orders
GROUP BY restaurant_id;
```

13. List users who have given reviews.
```
SELECT DISTINCT u.*
FROM Users u
JOIN Reviews r ON u.user_id = r.user_id;
```

14. Find the highest priced item in the Menu table.
```
SELECT * FROM Menu
WHERE price = (SELECT MAX(price) FROM Menu);
```

15. Show details of orders along with user names (JOIN).
```
SELECT o.*, u.name
FROM Orders o
JOIN Users u ON o.user_id = u.user_id;
```

16. Display order details with item names and quantity.
```
SELECT od.order_id, m.item_name, od.quantity
FROM OrderDetails od
JOIN Menu m ON od.menu_id = m.menu_id;
```

17. Find the total number of orders placed for each restaurant.
```
SELECT restaurant_id, COUNT(*) AS total_orders
FROM Orders
GROUP BY restaurant_id;
```

18. Show restaurants that have not received any reviews.
```
SELECT r.*
FROM Restaurants r
LEFT JOIN Reviews rv ON r.restaurant_id = rv.restaurant_id
WHERE rv.restaurant_id IS NULL;
```

19. Retrieve users who have not placed any orders.
```
SELECT u.*
FROM Users u
LEFT JOIN Orders o ON u.user_id = o.user_id
```

```
    WHERE o.user_id IS NULL;
```

20. Find the most ordered item based on quantity.

```
    SELECT m.item_name, SUM(od.quantity) AS total_quantity
    FROM OrderDetails od
    JOIN Menu m ON od.menu_id = m.menu_id
    GROUP BY m.item_name
    ORDER BY total_quantity DESC
    LIMIT 1;
```